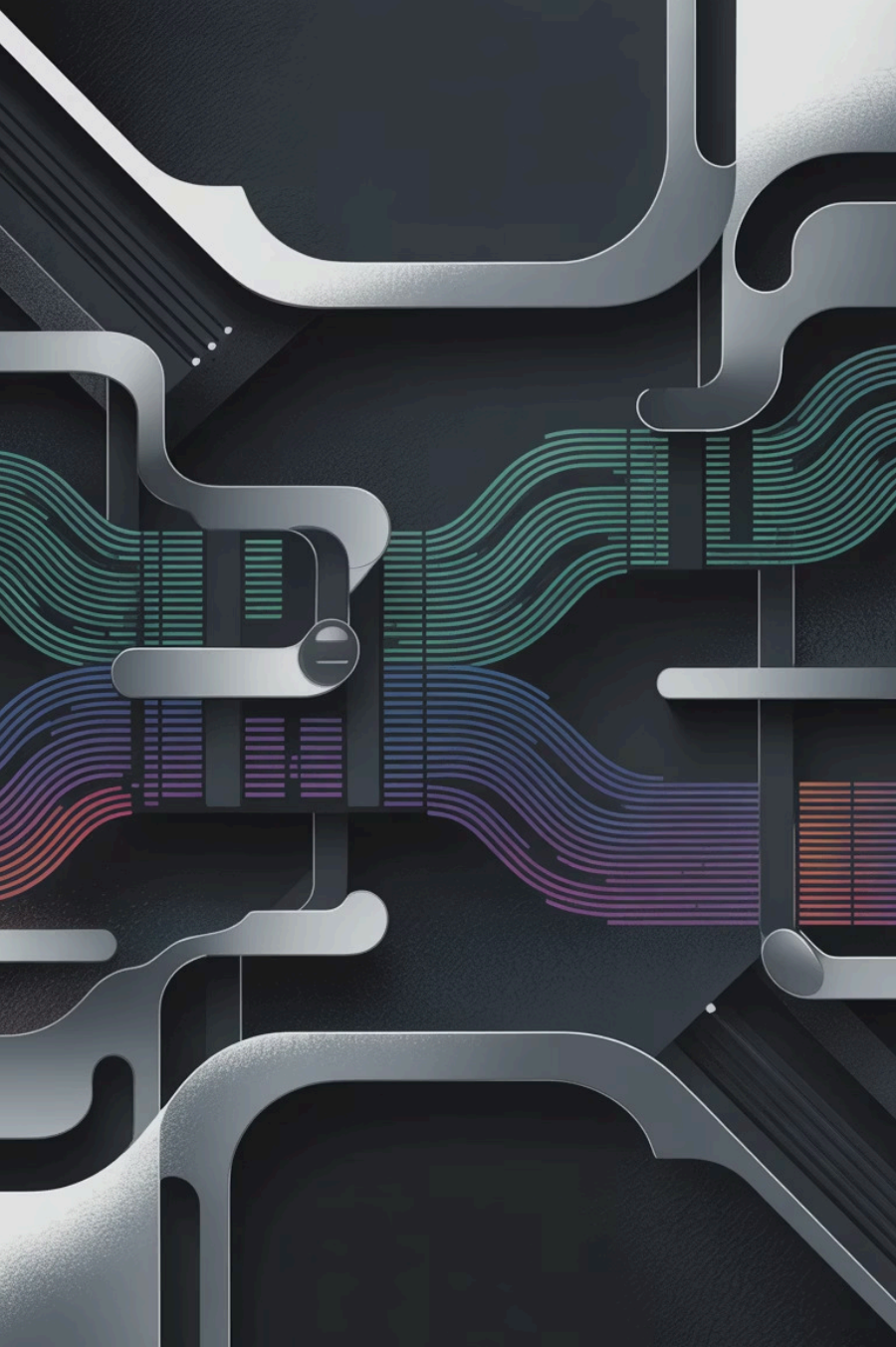




# Module 03 – Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Redux

# Data Engineering for LLM Pipelines

01

---

## Extract

Pull data from SQL databases and HTTP APIs using Python

03

---

## Load

Prepare datasets in formats compatible with LLM training and fine-tuning

02

---

## Transform

Clean, join, and validate data for quality and consistency

04

---

## Validate

Ensure data quality through schema validation and profiling

# HTTP API Fundamentals

## What You'll Learn

- Making GET requests with parameters
- Handling authentication securely
- Processing HTTP status codes
- Parsing JSON responses
- Implementing pagination strategies
- Building resilient retry logic

## Why It Matters

Modern LLM pipelines require data from diverse sources. Web APIs provide access to real-time information, social media content, knowledge bases, and structured datasets that enrich your training data.

Building reliable API clients ensures your data pipelines run smoothly in production, handling errors gracefully and respecting service limits.



# Understanding HTTP APIs

APIs (Application Programming Interfaces) let applications communicate over the web using HTTP protocols. RESTful APIs are the most common, using standard HTTP methods like GET, POST, PUT, and DELETE to interact with resources.

For data extraction, we primarily use GET requests to retrieve information. APIs return structured data, typically in JSON format, which Python can easily parse and process for downstream LLM workflows.



# Your First API Request

## The requests Library

Python's `requests` library simplifies HTTP communication. It handles connection management, encoding, and response parsing automatically.

Install it using pip if not already available:

```
pip install requests
```

This single library provides everything needed for production-grade API clients.

## Basic GET Request

```
import requests

response = requests.get(
    'https://api.example.com/data'
)

print(response.status_code)
print(response.json())
```

This minimal example demonstrates the core pattern: make a request, check the status, and parse the response.

# Working with Query Parameters

Query parameters filter, sort, or customize API responses. They appear in URLs after a question mark, like `?name=value&page=2`. The requests library handles parameter encoding automatically.

## Manual URL Construction

```
url = 'https://api.example.com/users?role=admin&status=active'
response = requests.get(url)
```

This works but requires manual encoding of special characters and can be error-prone.

## Using the params Argument

```
params = {
    'role': 'admin',
    'status': 'active',
    'limit': 100
}
response = requests.get(
    'https://api.example.com/users',
    params=params
)
```

**Best practice:** Always use the params dictionary for cleaner, safer code.

# Handling Authentication

## API Keys

Most common method. Pass keys in headers or query parameters.

```
headers = {  
    'Authorization': 'Bearer YOUR_KEY'  
}  
response = requests.get(url,  
    headers=headers)
```

## Basic Auth

Username and password encoded in headers. Use the auth parameter.

```
from requests.auth import  
HTTPBasicAuth  
  
response = requests.get(  
    url,  
    auth=HTTPBasicAuth('user', 'pass')  
)
```

## OAuth Tokens

Token-based authentication for secure, delegated access.

```
headers = {  
    'Authorization': f'Bearer {token}'  
}  
response = requests.get(url,  
    headers=headers)
```

❏ **Security tip:** Never hardcode credentials. Use environment variables or secure vaults like AWS Secrets Manager, Azure Key Vault, Hashi Corp Vault, etc.

# Understanding HTTP Status Codes

Status codes indicate whether your request succeeded and what happened. Production code must check these codes and respond appropriately.



## 2xx Success

- **200 OK:** Request succeeded, data returned
- **201 Created:** New resource created successfully
- **204 No Content:** Success but no data to return



## 4xx Client Errors

- **400 Bad Request:** Invalid parameters or malformed request
- **401 Unauthorized:** Missing or invalid authentication
- **404 Not Found:** Resource doesn't exist
- **429 Too Many Requests:** Rate limit exceeded



## 5xx Server Errors

- **500 Internal Server Error:** Server-side problem
- **502 Bad Gateway:** Upstream service failed
- **503 Service Unavailable:** Temporary outage

# Checking Response Status

## The `raise_for_status()` Method

```
response = requests.get(url)
response.raise_for_status()
data = response.json()
```

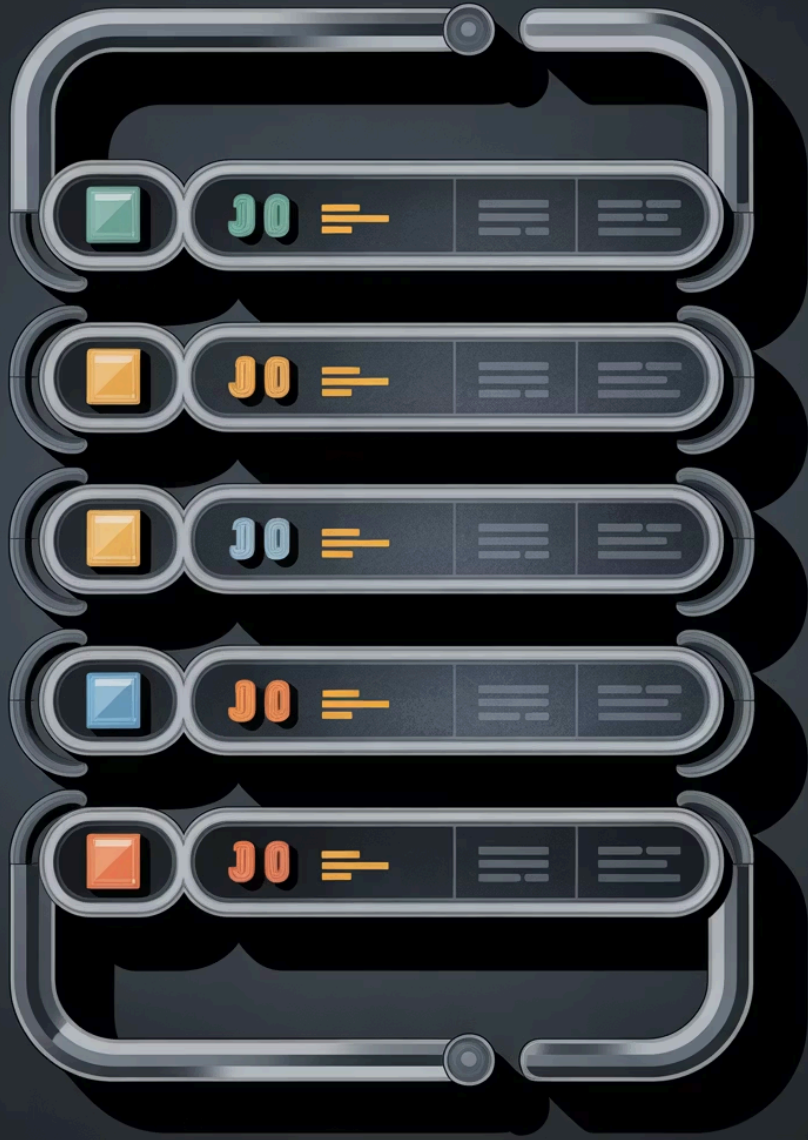
This method raises an exception for 4xx and 5xx status codes, stopping execution immediately. It's the simplest way to handle errors.

## Manual Status Checking

```
response = requests.get(url)

if response.status_code == 200:
    data = response.json()
elif response.status_code == 404:
    print("Resource not found")
elif response.status_code == 429:
    print("Rate limit hit")
else:
    print(f"Error: {response.status_code}")
```

Provides fine-grained control for custom error handling logic.



# Parsing JSON Responses

APIs typically return data in JSON format, which Python's requests library parses automatically using the `response.json()` method. This converts JSON into native Python dictionaries and lists.

```
response = requests.get('https://api.github.com/users/octocat')
user_data = response.json()
```

```
print(user_data['login'])      # Access top-level fields
print(user_data['public_repos']) # Get nested data
print(user_data['created_at'])  # Parse timestamps
```

Always check the status code before calling `.json()` to avoid parsing errors on failed requests. Handle potential JSON decode errors when APIs return malformed data.



# Real-World Example: GitHub API

Let's fetch repository information from GitHub's public API. This example demonstrates parameters, headers, and JSON parsing in a single workflow.

```
import os
import requests

# Auth: a token raises your core limit to 5,000/hr. Without one, you run
# unauthenticated (lower limits) — but you still get a valid request.
token = os.environ.get("GITHUB_TOKEN")
headers = {
    "Accept": "application/vnd.github+json",    # modern media type
    "X-GitHub-API-Version": "2022-11-28",      # date-based API versioning
}
if token:
    headers["Authorization"] = f"Bearer {token}" # 'Bearer' now preferred over 'token'

# NOTE: /search/* is limited to 30 requests/minute, not 5,000/hour
params = {
    "q": "language:python stars:>1000",
    "sort": "stars",
    "order": "desc",
    "per_page": 10,
}

response = requests.get(
    "https://api.github.com/search/repositories",
    headers=headers,
    params=params,
    timeout=(5, 30),    # (connect, read) — never omit
)
response.raise_for_status()
repos = response.json()

for repo in repos["items"]:
    print(f"{repo['name']}: {repo['stargazers_count']} stars")
```

# Implementing Pagination

APIs limit response sizes to prevent overload, returning data in pages. Large datasets require multiple requests to fetch all records. Understanding pagination patterns is essential for complete data extraction.

## Offset-Based

Skip N records, return next batch

```
?offset=0&limit=100  
?offset=100&limit=100
```

## Cursor-Based

Use tokens for next page

```
?cursor=abc123  
?cursor=def456
```

1

2

3

4

## Page-Based

Request specific page numbers

```
?page=1&per_page=100  
?page=2&per_page=100
```

## Link Headers

API provides next page URLs

```
Link: <url?page=2>; rel="next"
```

# Pagination Implementation Pattern

## Page-Based Pagination Loop

```
all_data = []
page = 1
per_page = 100

while True:
    params = {
        'page': page,
        'per_page': per_page
    }

    response = requests.get(url, params=params)
    response.raise_for_status()

    data = response.json()

    if not data: # No more results
        break

    all_data.extend(data)
    page += 1

print(f"Fetched {len(all_data)} total records")
```

## Key Considerations

- **Empty check:** Stop when API returns no results
- **Rate limiting:** Add delays between requests
- **Progress tracking:** Log page numbers for debugging
- **Error handling:** Retry failed pages
- **Data accumulation:** Store results efficiently



Always respect API documentation for maximum page size limits.



# Understanding Rate Limits

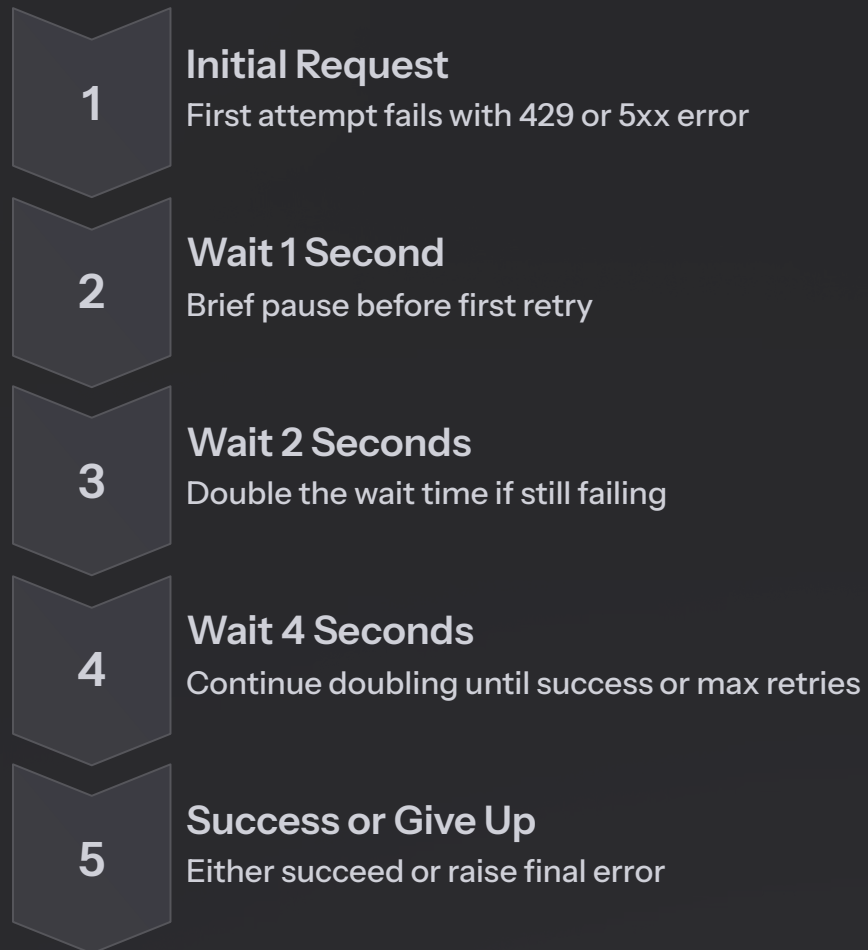
APIs enforce rate limits to prevent abuse and ensure fair access. GitHub uses **separate buckets**: core REST endpoints allow **5,000 requests/hour** when authenticated (60/hour unauthenticated per IP), while the **Search API is far stricter** — **30 requests/minute** (code search is 10/minute).

|                                                        |                                               |                                          |
|--------------------------------------------------------|-----------------------------------------------|------------------------------------------|
| 5000                                                   | 60                                            | 429                                      |
| Requests per hour                                      | Requests per hour                             | HTTP Status Code                         |
| Typical rate limit for authenticated GitHub API access | Unauthenticated GitHub API limit (much lower) | Indicates you've exceeded the rate limit |

Check response headers like `X-RateLimit-Remaining` and `X-RateLimit-Reset` to monitor your usage proactively.

# Exponential Backoff Strategy

When requests fail due to rate limits or temporary errors, exponential backoff provides an intelligent retry mechanism. Instead of immediately retrying, wait progressively longer between attempts.



This approach prevents overwhelming struggling services while giving transient errors time to resolve.

# Implementing Exponential Backoff

```
import time
import random
import requests

RETRYABLE_STATUS = {429, 500, 502, 503, 504} # 504 added (Gateway Timeout)

def fetch_with_backoff(url, max_retries=5, initial_delay=1, **kwargs):
    """GET a URL with exponential backoff + jitter on transient failures."""
    kwargs.setdefault("timeout", (5, 30)) # (connect, read)

    for attempt in range(max_retries):
        delay = None
        try:
            response = requests.get(url, **kwargs)

            if response.ok: # any 2xx succeeds (not just 200)
                return response.json()

            rate_limited_403 = (
                response.status_code == 403
                and response.headers.get("X-RateLimit-Remaining") == "0"
            )
            if response.status_code in RETRYABLE_STATUS or rate_limited_403:
                delay = _retry_delay(response, attempt, initial_delay)
            else:
                response.raise_for_status() # other 4xx: fail fast, no retry

        except (requests.exceptions.Timeout,
                requests.exceptions.ConnectionError): # catch connection drops too
            delay = initial_delay * (2 ** attempt) + random.uniform(0, 1)

        if delay is not None and attempt < max_retries - 1: # don't sleep after last try
            print(f"Attempt {attempt + 1} failed. Retrying in {delay:.1f}s...")
            time.sleep(delay)

    raise RuntimeError(f"Failed after {max_retries} attempts")

def _retry_delay(response, attempt, initial_delay):
    """Honor the server's Retry-After header when present; else exponential + jitter."""
    retry_after = response.headers.get("Retry-After", "")
    if retry_after.isdigit():
        return int(retry_after)
    return initial_delay * (2 ** attempt) + random.uniform(0, 1)
```

# Best Practices for Production API Clients

## Secure Credentials

- Use environment variables
- Never commit API keys to git
- Rotate tokens regularly
- Use secrets management services

## Handle Errors Gracefully

- Check status codes always
- Implement retry logic
- Log failures for debugging
- Fail fast on client errors

## Monitor and Log

- Track rate limit usage
- Log request timestamps
- Monitor success rates
- Alert on repeated failures

## Optimize Performance

- Use connection pooling
- Cache responses when appropriate
- Batch requests if supported
- Set reasonable timeouts

# Common Pitfalls to Avoid

## ✗ Ignoring Rate Limits

Hammering APIs without backoff leads to blocks and failed pipelines. Always implement rate limit detection and exponential backoff.

## ✗ Not Validating Responses

Assuming API responses match expected schemas causes crashes. Validate structure and handle missing fields defensively.

## ✗ Hardcoding URLs and Credentials

Makes code inflexible and insecure. Use configuration files and environment variables for all external dependencies.

## ✗ Missing Timeout Configuration

Requests without timeouts can hang indefinitely. Always set reasonable timeout values for both connection and read operations.





Thanks